A Bit-Parallel Algorithm to Solve the κ-Nearest Neighbor String Matching Problem

Chia Shin Ou and R. C. T. Lee

Department of Computer Science
National Tsing Hua University, Hsinchu, Taiwan
d9662836@oz.nthu.edu.tw, rctlee@ncnu.edu.tw

Abstract

The nearest neighbor string matching problem is defined as follows: We are given a text string T and a pattern string P. Our problem is to find a substring in T whose edit distance with P is the smallest among all substrings in T. This problem can be solved by the Seller's approach which finds the substring ending at location i of T whose edit distance with P is the smallest. Of course, we may use Myers' approach which is an improved version of Seller's algorithm. But, this approach needs to store the entire dynamic programming matrix. To improve Myers' approach, we proposed floating window and circular approaches. The κ-nearest neighbor string matching problem is defined as follows: Given a text string T and a pattern string P, find all substrings of T whose edit distances with P are between the smallest and the κ-th among all substrings of T. We may use Hyyro and Novarro's approach to solve the κ-nearest neighbor string matching problem. But, their algorithm was designed with an error bound k in mind. To overcome this problem, we used again the floating window approach. Experimental results show that our algorithms are efficient.

1 Notations and Definitions

In this paper, we are interested in processing strings. A string S is a sequence of characters, denoted as $S = s_1 s_2 \cdots s_n$ where each s_i is a character. We shall use $S(i, j)$ to denote $s_i s_{i+1} \cdots s_j$. A suffix of a string $s_1 s_2 \cdots s_n$ is a substring $S(i, n)$ where $1 \le i \le n$.

The edit distance between two strings A and B, denoted $ED(A, B)$ is the smallest number of deletions, insertions and substitutions to transform A to B [5]. The nearest neighbor string matching (NNSM) problem is defined as follows: Given a text string $T = t_1 t_2 \cdots t_n$ and a pattern string $P = p_1 p_2 \cdots p_m$, find all substrings of T whose edit distances with P are the smallest among all substrings of T. Given T = AACTCATCAA and P = ATCTCA, the nearest neighbor strings are AACTCA and ATCTA.

2 The Seller's Algorithm

NNSM problem can be solved by Seller's Algorithm [5]. This algorithm solves the following problem: Given a text string $T = t_1 t_2 \cdots t_n$ and a pattern string $P = p_1 p_2 \cdots p_m$, for each location i of T, for all $1 \le i \le n$, find the suffix S_i of $T(1, i)$ such that $ED(S_i, P)$ is the smallest. Let $S_{i,j}$ be the suffix of $T(1, i)$ such that $ED(S_{i,j}, P(1, j))$ is the smallest for all $1 \le j \le m$. To simplify the notation, let $c(i, j)$ denote $ED(S_{i,j}, P(1, j))$. The Seller's Algorithm is based upon dynamic programming [7] which can be found by the following recursive formula.

$$c(i, j) = \begin{cases} c(i-1, j-1) & \text{if } p_j = t_i \\ 1 + \min\{c(i-1, j), c(i, j-1), c(i-1, j-1)\} & \text{else} \end{cases} \quad (1)$$

The initial condition of recursive formula (1) is that $c(0, j) = j$ and $c(i, 0) = 0$ for $0 \le i \le n$ and $1 \le j \le m$.

As can be seen, the algorithm essentially produces a dynamic programming table. The following is an example of the table produced.

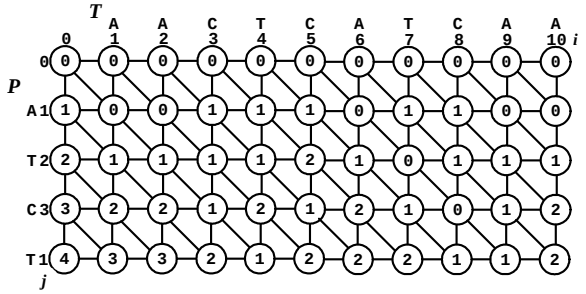

Figure 1. A dynamic programming matrix of Seller's algorithm with T = AACTCATCAA and P = ATCT.

In the above table, we can see that for locations 4, 8 and 9 of T, there are substrings ending at these

locations which will give the solutions of our nearest neighbor string matching problem. But, we still do not know which substrings they are. To find the substrings, we must perform a backtracking search. The following is a typical such backtracking algorithm.

Backtracking algorithm

Input: Dynamic programming table of Sellers' algorithm
Output: All nearest neighbor strings for the NNSM problem

if $p_j = t_i$, **then** trace back to $c(i-1, j-1)$;
else
    trace back to all min{$c(i–1, j)$, $c(i, j–1)$, $c(i–1, j–1)$};
end if

Using this algorithm, we can find all the nearest neighbor substring ending at location t_8.

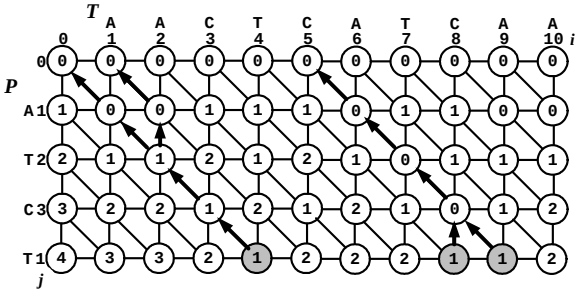

Figure 2. Backtracking from the dynamic programming matrix with T = AACTCATCAA and P = ATCT. The nearest neighbor strings are AACT, ACT, CATC and CATCA.

This backtracking requires memory space $O(mn)$ to store the dynamic programming matrix. This can be quite expensive because n can be very large. For instance, in our experiments later demonstrated, n is 10^9. Myers' algorithm [3] is one of efficient way to obtain the dynamic programming matrix by using the bit-parallel approach. We introduce the Myers' algorithm in Section 3

3 Myers' Algorithm with Backtracking Algorithm

3.1 Introduction of Myers' algorithm

Myers' algorithm [3] is to fill the dynamic programming matrix by using the bit-parallel approach based on [5]. Formally, it defines the dynamic programming matrix's vertical value $\Delta v(i, j) = c(i, j) - c(i, j-1)$, the horizontal value $\Delta h(i, j) = c(i, j) - c(i-1, j)$, and diagonal value $\Delta d(i, j) = c(i, j) - c(i-1, j-1)$. Their range of values comes from the properties of the dynamic programming matrix [2, 4].

Lemma 1 [2, 4]. For all i and j, $\Delta v(i, j)$, $\Delta h(i, j) \in \{-1,0,1\}$ and $\Delta d(i, j) \in \{0,1\}$.

Consider that the following Boolean variables:

$$vp(i, j) \equiv \Delta v(i, j) = +1 \quad (2)$$
$$vn(i, j) \equiv \Delta v(i, j) = -1 \quad (3)$$
$$hp(i, j) \equiv \Delta h(i, j) = +1 \quad (4)$$
$$hn(i, j) \equiv \Delta h(i, j) = -1 \quad (5)$$
$$d0(i, j) \equiv \Delta d(i, j) = 0 \quad (6)$$

The Boolean vectors $HN(j)$, $VN(j)$, $HP(j)$, $VP(j)$, and $D0(j)$ are vectors indexed by i, which change their values for each new text position j, as we traverse the text. In preprocessing step, given a string $P = p_1p_2…p_m$ and a character x, the incidence vector of x on P, denote as $EQ(x)$, is $(eq_1, eq_2, …, eq_m)$ where $eq_i = 1$ if $a_i = x$ and $eq_i = 0$ otherwise. The most important value in the dynamic programming matrix is the last value $c(m, j)$ in the column j of the dynamic programming matrix. The solutions can be determined by checking whether $c(m, j)$ is smaller than the error bound for all j or not. Hence, the value *score* stores $c(m, j)$ and is updated by using $HP(m, j)$ and $HN(m, j)$. The following is the Myer's Algorithm. In the pseudo-code, we use C-like notation for bit operations, e.g., '&' denotes bit-wise AND, '|' denotes bit-wise OR, '^' denotes bit-wise XOR, and '<<' denotes shifting a bit-vector to the left. The shift is assumed to use zero filling. For instance, $11100=1^3 0^2$.

Algorithm 1: Myers Algorithm

Input: $T = t_1t_2 \cdots t_n$, $P = p_1p_2 \cdots p_n$ and an error bound k
Output: Five matrices containing VP, VN, HP, HN and $D0$ defined in Equations (2) to (6)

Preprocessing of *EQ*;
$VP(0) = 1^m$; $HP(0) = 0$;
score = m;
for $i = 1$ **to** n
    $X = EQ(t_i) \mid VN(i-1)$;
    $D0(i) = ((VP(i-1)+(X \,\&\, VP(i-1)))\hat{}\, VP(i-1)) \mid X$;
    $HN(i) = VP(i-1) \,\&\, D0(j)$;
    $HP(i) = VN(i-1) \mid \sim(VP(i-1) \mid D0(i))$;
    $X = HP(i)<<1$
    $VN(i) = (X \,\&\, D0(j))$;
    $VP(i) = (HN(i) <<1) \mid \sim(X \mid D0(i))$;
    if $HP(i) \,\&\, 10^{m-1} \neq 0$ **then**
        score = *score* + 1;
    else if $HN(i) \,\&\, 10^{m-1} \neq 0$ **then**
        score = *score* − 1;

end for

3.2 Algorithm 1: Our bit-parallel algorithm to solve the NNSM problem

3.2.1 Floating window and the cyclic approach

When the nearest neighbor string matching (NNSM) problem is solved by using [3], the entire dynamic programming table needs to be stored to construct the backtracking paths. Hence, the space complexity is O(*mn*/*w*). Note that for our case, the edit distance between a nearest neighbor of pattern *P* cannot be greater that *m*, the size of *P*. Therefore we start with a floating window size of 2*m*. At any time, if we find a new nearest neighbor with edit distance *sd*, we reduce the floating window size to be *m*+*sd*.

We still face one problem. Consider Figure 3 which shows the window starting form location *i* and ending at location *i*+*m*+*sd*–1. Suppose all of the data in this window have been obtained, and we will have a new window starting at *i*+1 and ending at *i*+*m*+*sd*.

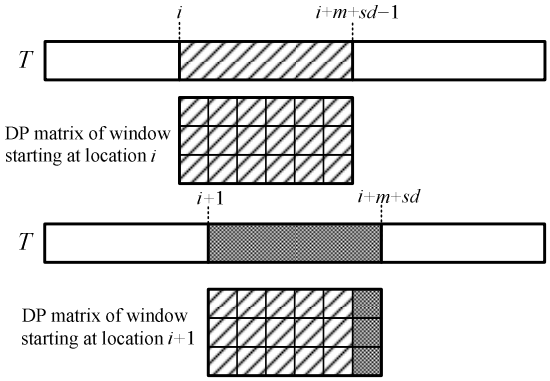

Figure 3. DP matrix of window starting at location *i* and DP matrix of a new window starting at location *i*+1.

We therefore have two choices: (1) Use the old memory space and move the data one column to the left. (2) Declare a new memory space for this new window. We have a mechanism which avoids this new memory declaration. We can avoid this because of the following observations:

(1) Any column *i* of the dynamic programming table depends upon the data in the column $i-1$.
(2) The starting column of the old window, namely column *i*, will not be used any more in the new window.

Based upon the above observations, we may calculate the data in column *i*+*m*+*sd* based upon the data in column *i*+*m*+*sd*–1 and then store the data in column *i*. In reality, to start with, we only declare a memory space with maximum size 2*m* and reduce it to *m*+*sd*. In other words, we never have to declare any new memory space allocation. We call this the *circular approach*. *VP*, *VN*, *HP*, *HN* and *D*0 tables are stored by using this circular method. For *VP*(*i*), it will be stored in the $VP(i \bmod (m+sd))$. Similarly, the backtracking algorithm will also trace by the circular way which is shown in Figure 4.

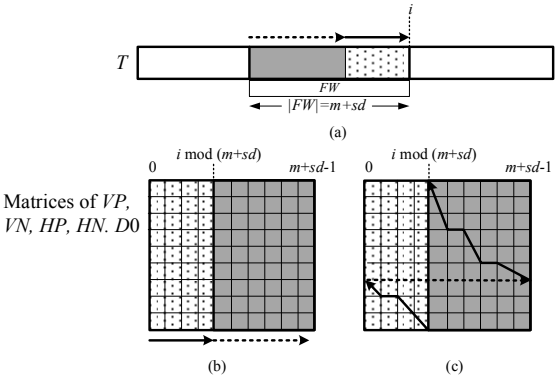

Figure 4. (a) The floating window *FW* in *T* (b) *VP*, *VN*, *HP*, *HN*, and *D*0 matrices are saved by using the circular approach. (c) Backtracking algorithm is worked by using the circular approach.

3.3 Our bit-parallel algorithm with backtracking algorithm

In our bit-parallel algorithm, we do not store the original DP matrix, so the traditional backtracking algorithm can not be used in matrices obtained by bit-parallel algorithm. However, the information are still contained in Boolean matrices of *VP*, *VN*, *HP*, *HN*, and *D*0. For instance, given *T* = AACTCATCTA and *P* = ATCT, *VP*, *VN*, *HP*, *HN*, and *D*0 are constructed in Figure 5 in where window starting at location 1.

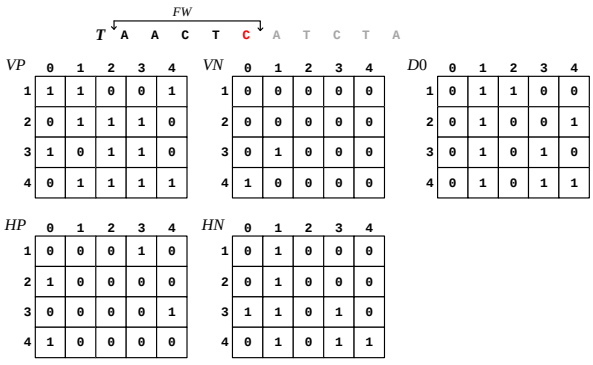

Figure 5. Boolean matrices of *VP*, *VN*, *HP*, *HN*, and *D*0 with *T* = AACTCATCTA and *P* = ATCT in window starting at location 1.

In backtracking algorithm, if $t_i = p_j$ for $1 \le i \le n$ and $1 \le j \le m$, the diagonal is the only way to trace back. If $t_i \neq p_j$, we have to use the matrices of *VP*, *VN*, *HP*, *HN*, and *D*0 to construct the backtracking paths. We need to consider the following cases.

If one of $d0(i, j) = 0$, $vp(i, j) = 1$ and $hp(i, j) = 1$ is true, we consider:

(1) $d0(i, j) = 0$: according to [6], the diagonal value is the minimal value, and the diagonal way needs to be traced back.
(2) $vp(i, j) = 1$ (or $hp(i, j) = 1$): the vertical (horizontal) value is the minimal value, and the vertical (horizontal) way needs to be traced back.

Otherwise, we consider whether $d0(i, j)$, $\sim (vp(i, j)\ and\ (vn(i, j)))$ or $\sim (hp(i, j)\ and\ (hn(i, j)))$ is true. If one of them is true, there are three cases shown in the following:

(3) $d0(i, j) = 1$, we trace back to the diagonal.
(4) $vp(i, j)\ and\ (vn(i, j)) = 0$, the vertical needs to trace back.
(5) $hp(i, j)\ and\ (hn(i, j)) = 0$, the horizontal needs to trace back.

According to these five cases, the backtracking algorithm for our bit-parallel algorithm can be constructed shown in the following.

Algorithm 2 : Backtracking Algorithm for Our Bit-parallel Algorithm

Input: The *VP*, *VN*, *HP*, *HN* and *D*0 tables.
Output: All the nearest neighbor strings of *P*.

```
BACKTRACK(i, j, x_time)
x = i mod (m + sd);
if x = 0 then
   y = m+sd−1;
else
   y = x−1;
end if
   if j = 0 or x_time = 0 then
      Return TRUE;
else if p_j = t_i then
   BACKTRACK(y, j−1, x_time−1);
else
   if vp(x, j) | hp(x, j) |~ d0(x, j) then
      if vp(x, j) = 1 then
         BACKTRACK(i, j−1, x_time);
      end if
      if hp(x, j) = 1 then
         BACKTRACK(y, j, x_time−1);
      end if
      if d0(x, j) = 0 then
         BACKTRACK(y, j−1, x_time−1);
      end if
      else
         if d0(x, j) |~ (vp(x, j) & vm(x, j) |
         ~ (hp(x, j) & hm(x, j))) then
            if d0(x, j) = 1 then
               BACKTRACK(y, j−1, x_time−1);
            endif
            if (vp(x, j) & vm(x, j)) = 0 then
               BACKTRACK(i, j−1, x_time);
            end if
            if (hp(x, j) & hm(x, j)) = 0 then
               BACKTRACK(y, j, x_time−1);
            end if
         end if
      end if
   end if
end if
Return FALSE;
```

Once we find a new nearest neighbor that $score < sd$, algorithm 2 will be executed immediately. It is not hard to know *sd* = *m* for beginning. If *score* < *sd*, a new nearest neighbor string is found and we set *sd* = *score*. The NNSM problem based on [3] is shown in the following.

Algorithm 3: NNSM algorithm

```
Preprocessing of EQ;
VP(0) = 1^m; HP(0) = 0;
score = m;  sd = m;
for i = 1 to n
    x = i mod (m + sd);
    if (i mod (m + sd)) = 0 then
        y = m + sd − 1;
    else
        y = x − 1;
    end if
    X  = EQ(t_i) | VN(x);
    D0(x) = ((VP(y) + (X & VP(y))) ^ VP(y)) | X;
    HN(x) = VP(y) & D0(i);
    HP(x) = VN(y) | ~(VP(y) | D0(x));
    X = HP(x)<<1
    VN(x) = (X & D0(x));
    VP(x) = (HN(x) <<1) | ~(X | D0(x));
    if HP(x) & 10^(m-1) ≠ 0 then
        score = score + 1;
    else if HN(x) & 10^(m-1) ≠ 0 then
        score = score − 1;
    if score ≤ sd, then
        BACKTRACK(i, m, m);
        sd = score;
    end if
end for
```

3.4 κ-nearest neighbor string matching problem

The κ-nearest neighbor string matching (κ-NNSM) problem is defined as follows: Given a text string $T = t_1 t_2 \cdots t_n$ and a pattern string $P = p_1 p_2 \cdots p_m$, find all substrings of T whose edit distances with P are between the smallest and the κ-*th* among all substrings of T. The κ-NNSM problem also can be solved by Sellers' algorithm with backtracking algorithm. However, it will cost more time to find the nearest neighbor strings in backtracking algorithm. Hence, we propose a bit-parallel algorithm which can solve the κ-NNSM without backtracking algorithm in the next section. Our algorithm is based upon the HN Algorithm [1].

4 Algorithm 2: Modification of HN Algorithm

4.1 Introduction of HN algorithm

The Hyyro and Navarro's (HN) bit-parallel algorithm is used to find the approximate string matching problem which is defined in the following: Given a text string T, a pattern string P and an error bound k, find all substrings of T whose edit distance with P is less than or equal to k. There are two filtering ideas used in HN algorithm. The first one is used in the partial filter window to sieve out the unnecessary searching positions in T, and the second one is used in the searching window to avoid computing the whole DP matrix. In the following, we introduce the first filter [10, 11].

4.1.1 The first filter of HN algorithm

Proposition 1 [2]: Given a partial filter window W_f in text T with length $m-k$, if there exists a suffix S at the position j of T in W_f such that the edit distance between S and any substring of P is larger than k, then we can move P to the position j+1 of T; otherwise, we shift P by one step.

According to Proposition 1, if there is a suffix S beginning at the position j of T such that the edit distance between S and any substring of P is larger than k, it means that values in column j of DP matrix are larger than k. [1] used a backward scanning algorithm which can determine whether there exists a suffix S such that no substring P' in P has the property that $ED(S, P') \le k$. Thus, we can shift the window as early as possible. It is difficult to check all values in the column j of DP matrix in linear time, so [1] proposed an idea to determine that some specific positions of witnesses' values in the column j of DP matrix are larger than k' or not, and these witnesses' values have the same interval distance Q. Let k' be $k + \lfloor Q/2 \rfloor$ and $t = \lceil m/Q \rceil$ be consecutive witnesses into vector MC. In order to determine Q, the following lemma is described.

Lemma 2 [1]: $Q = \lceil \log_2(m-k+1) \rceil$ if $m - k - k' \le 2^{Q-1}$ and $k'+1 \le 2^{Q-1}$; otherwise, $Q + 1$.

For updating all of the witness values, each witness $c(m-rQ, j)$ can be updated by considering the $(m-rQ)$-th bits of HP and HN. That is, it defines a mask $sMask = (0^{Q-1}1)^t 0^{m+Q-1-tQ}$ and updates all witnesses in parallel by the following formula.

$$MC = MC + (HP \,\&\, sMask) - (HN \,\&\, sMask) \quad (7)$$

For determining that all witnesses exceed k', we store each witness with extra value $b = 2^{Q-1} - 1 - k'$. That is, if $c(m-rQ, j) = x$, the corresponding witness stores the value $x + b$. Hence, $MC = [b]_Q^t 0^{m+Q-1-tQ}$ initially. By this way, if the Q-th bit of a witness is equal to 1, the corresponding value in DP matrix is larger than k'. Hence, it defines $eMask = (10^{Q-1})^t 0^{m+Q-1-tQ}$ to check whether the Q-th bits of all witnesses are equal to 1 or not by using the following formula.

$$MC \,\&\, eMask = eMask \quad (8)$$

Thus, it can stop the scanning of W_f whenever the Equation (8) is true. In the searching phase, it is done by using the bit-parallel algorithm which is modified by [3]. In the following, we introduce the modification bit-parallel algorithm.

4.1.2 The modification of Myers' algorithm for computing edit distance with the second filter

The modified version of Myers' algorithm [1] finds the edit distance through a bit-parallel approach shown in the following. It used the bits of the computer word(w) with a worst case of $O(mn/w)$ time. Through the following formulas, the edit distance can be found in a bit-parallel way.

Algorithm 2: MyersStep($EQ(t_j)$)

$X = EQ(t_j) \mid VN(j-1)$;
$D0(j) = ((VP(j-1) + (X \,\&\, VP(j-1))) \wedge VP(j-1)) \mid X$;
$HN(j) = VP(j-1) \,\&\, D0(j)$;
$HP(j) = VN(j-1) \mid \sim(VP(j-1) \mid D0(j))$;
$X = (HP(j) << 1) \mid 0^{m-1}1$;
$VN(j) = (X \,\&\, D0(j))$;
$VP(j) = (HN(j) << 1) \mid \sim(X \mid D0(j))$;

The second filtering approach is based on the

idea of diagonal property [2] which is proposed in [6], and it is used in the searching window. We show the diagonal property in the following.

Lemma 3 [11, 15]: For all i and j, values $c(i, j)$ of DP matrix in the diagonal d where $d = i - j$ are non-decreasing.

The lemma 3 allows us to know the largest l such that $c(i, l) \le k$ for each i. Clearly, l exists if and only if $c(i, j) \le k$ for some j. Hence, we maintain a witness in the current column i of the DP matrix. This will tell us the last cell which does not exceed k. Initially, since $c(0, j) = 0$, we set $l = k$. Since $c(i, l+1) - c(i-1, l) = \Delta d(i,l+1) \in \{0, 1\}$, l is increased by 1 if and only if the bit $D0(i, l)=1$. If $D0(i, l)=0$, l will remain the same value or to be decreased. If we reach $l = 0$ and still $c(i,l)>k$, then all the rows are larger than k and we stop the scanning process. If $l = m$, we have $c(i, m)\le k$ and found an occurrence at location i in T. According to lemma 3, the filtering approach used in Myers algorithm is presented as follows.

Algorithm 3: Myers' Bit-Parallel Algorithm BPA(EQ, W_s, k) with the Filtering Approach

Input: The Substring window W_s, pattern P and the error bound k
Output: All substrings in W_s whose edit distance is smaller than or equal to k

$VP(0) = 1^m$, $VN(0) = 0^m$;
$l = 0^{m-k}10^{k-1}$;
for $j = 1$ **to** n
  BPMStep(t_j);
  $l = l << 1$;
  if $D0$ & $l = 0^m$
    $val = k + 1$;
    while $val > k$
      if $l = 0^{m-1}1$
        return *FALSE*;
      end if
      if VP & $l \neq 0^m$
        $val = val - 1$;
      end if
      if VN & $l \neq 0^m$
        $val = val + 1$;
      end if
      $l = l >> 1$;
    end while
  else if $l = 10^{m-1}$
    return *TRUE*;
  end if
  $j = j + 1$;
return *FALSE*;

The full HN Algorithm is shown in the following.

Algorithm 4: HN Bit-Parallel Algorithm

Input : A text T, a pattern P and the error bound k
Output : All substrings in T whose edit distance is smaller than or equal to k

//Preprocessing
for $c \in \Sigma$
  $Bf[c] = 0^m$, $Bb[c] = 0^m$;
for $i = 1$ **to** m
  $Bf[P_i] = Bf[P_i] \mid 0^{m-i}10^{i-1}$;
  $Bb[P_i] = Bb[P_i] \mid 0^{i-1}10^{m-i}$;
  $Q = \lceil \log 2(m-k+1) \rceil$;
  if $2^{Q-1} < \max(m - 2k - \lfloor Q/2 \rfloor, k + 1 + \lfloor Q/2 \rfloor)$
    $Q = Q + 1$;
  end if
  $b = 2^{Q-1} - k - \lfloor Q/2 \rfloor - 1$
  $t = \lfloor m/Q \rfloor$
  $sMask = (0^{Q-1}1)^t 0^{m+Q-1-tQ}$
  $eMask = (10^{Q-1})^t 0^{m+Q-1-tQ}$
  //Searching
  for $pos = 0$ **to** $n - (m - k)$
    $j = m - k$, $last \leftarrow m - k$;
    $VP = 0^m$, $VN \leftarrow 0^m$;
    $MC = [b]_Q^t 0^{m+Q-1-tQ}$;
    while $j \neq 0$ & $((MC \,\&\, eMask) \neq eMask)$
      MyersStep(t_{pos+j})
      $MC = MC + (HP \,\&\, sMask) - (HN \,\&\, sMask)$
      $j = j - 1$
      if $MC \,\&\, 10^{m+Q-2} = 0^{m+Q-1}$
        if $j > 0$
          $last \leftarrow j$
        end if
      else if
        BPA(Bf, $t_{pos+1 \cdots n}$, k)
      end if
      report an occurrence at $pos + 1$
      $pos \leftarrow pos + last$
    end while
  end for
end for

In the next section, we present our bit-parallel approach to solve the κ-nearest neighbor string matching problem (κ-NNSM). In [1], there is an error bound interval. In the κ-NNSM problem, there is no error bound k. In [1], for the searching phase, the size of the window is fixed to be $m+k$. For the κ-NNSM problem, we don't have to fix the window size because there is no error bound k. That is, we can use a floating window size which is $m+sd$ where sd is the presently found smaller edit distance.

4.2 Our bit-parallel algorithm for κ-NNSM

Our proposed algorithm is composed of two phases: searching and filtering. The searching

phase is to find the nearest neighbor strings with the *floating searching window* FW_s (initial window's size is $2m$), and the filtering phase is used to determine whether the searching phase needs to be obtained or not with the *floating filtering window* FW_f. For solving the κ-NNSM problem, we have to relax all limitations of filters in HN algorithm. The first one is to increase the floating searching window size $|FW_s|$. If $sd+t \geq m$, $|FW_s| = 2m$; otherwise, $|FW_s| = m+sd+t-1$. This equation is shown as follows.

$$|FW_s| = m + min\{sd+t-1, m\} \quad (9)$$

For κ-NNSM, the limitation of l has to be relaxed to $min\{sd+t-1, m\}$. The modification algorithm of Algorithm 3 is shown in the following.

Algorithm 5: The Modification Algorithm MFBPA(EQ, FW_s, sd) with Floating Searching Window FW_s

Input: the EQ table corresponding to the pattern P, the floating searching window FW_s, and the edit distance sd.
Output: All the nearest neighbor strings in FW_s of P

$VP(0) = 1^m$, $VN(0) = 0^m$;
$l = 0^{m-min\{sd+t-1, m\}}10^{min\{sd+t-1, m\}-1}$;
for $i = 1$ **to** m
  $Bf(p_i) = Bf(p_i) \mid 0^{m-i}10^{i-1}$;
end for
for $j = 1$ **to** $m+ min\{sd+t-1, m\}$
  BPMStep($Bf(w_j)$);
  $l = l << 1$;
  if $D0$ & $l = 0^m$
    $td = sd + 1$;
    while $td > sd$
      if $l = 0^{m-1}1$
        return *FALSE*;
      end if
      if VP & $l \neq 0^m$
        $td = td - 1$;
      end if
      if VN & $l \neq 0^m$
        $td = td + 1$;
      end if
      $l = l >> 1$;
    end while
  else if $l = 10^{m-1}$
    return *TRUE*;
  end if
  $j = j + 1$;
return *FALSE*;

The size of FW_f also needs to be modified by the floating window approach as shown in the following.

$$|FW_f| = m - max\{sd-t+1, 0\} \quad (10)$$

For the filtering phase, the HN algorithm is used to percolate the unnecessary positions in T for searching by using the Lemma 2. In κ-NNSM problem, we set the extra value b shown as follows.

$$b = 2^{Q-1} - \lfloor Q/2 \rfloor - 1 - max\{sd - t+1, 0\} \quad (11)$$

According the above Equations (9), (10) and (11), we can solve the κ-NNSM problem by the following algorithm.

Algorithm 6: The Bit-Parallel Algorithm of the κ-NNSM Problem

Input: A text T, a pattern P and the nearest neighbor bound κ
Output: all nearest neighbor strings of P in T

***//Phase* 1**
$VP = 1^m$, $VN = 0^m$;
$l = 10^{m-1}$;
$td = m$, $sd = m$;
for $c \in \Sigma$
  $Bf(c) = 0^m$, $Bb(c) = 0^m$;
end for
for $i = 1$ **to** m
  $Bf(p_i) = Bf(p_i) \mid 0^{m-i}10^{i-1}$;
  $Bb(p_i) = Bb(p_i) \mid 0^{i-1}10^{m-i}$;
end for
MFBPA(Bf, $T_{1\cdots 2m}$, sd)
***//Phase* 2**
$Q = \lceil \log_2(m - sd + 1) \rceil$;
if $2^{Q-1} < \max(m - 2sd - \lfloor Q/2 \rfloor, sd + 1 + \lfloor Q/2 \rfloor)$
  $Q = Q + 1$;
end if
$b = 2^{Q-1} - \lfloor Q/2 \rfloor - 1 - max\{sd - t+1, 0\}$;
$t = \lfloor m/Q \rfloor$;
$sMask = (0^{Q-1}1)^t 0^{m+Q-1-tQ}$;
$eMask = (10^{Q-1})^t 0^{m+Q-1-tQ}$;
$pos = 0$;
while $pos \leq n - (m - max\{sd - t+1, 0\})$
  $j = m - max\{sd - t+1, 0\}$;
  $last = m - max\{sd - t+1, 0\}$;
  $VP = 0^m$, $VN = 0^m$;
  $MC = [b]_Q^t 0^{m-Q-1-tQ}$;
  while $j \neq 0$ **and** (MC & $eMask$) $\neq eMask$
    MyersStep($Bb(T_{pos+j})$);
    $MC = MC + (HP \& sMask) - (HN \& sMask)$;
    $j = j - 1$;
    if (MC & 10^{m+Q-2}) $= 0^{m+Q-1}$
      if $j > 0$
        $last = j$;
      end if
    else if MFBPA(Bf, $T_{pos+1\cdots m+sd}$, sd)
      store an occurrence at $pos + 1$;

end if
pos = *pos* + *last*;
end while
end while

5 Experiments

In Experiments 1 and 2, we compared our bit-parallel approach based on Myers for the NNSM problem against the [5]. The experiments were tested with random pattern and text and took the average time with 1000 times. With both the text and pattern, we use four alphabets (A, C, G, and T) because they behave like DNA sequence. And, we test Experiment 3 with the bioinformatics data: *Chlamydomonas* for the text string. The total elapsed time is measured for performance evaluations. The computer used in the experiments was a 32-bit Intel®Core™2 Quad CPU, 3 GB of RAM, and WinXP OS. All experiment programs were compiled with the DEV-C compiler. There were no other active significant processes running on the computer while performing the experiments.

[5] is presented by **Sel**, and our algorithm is presented by **OL** in the results. Figure 5 shows that the experiment result for σ =4 and $|T|=10^5$ with $|P|$=10, 20 and 30. This experiment shows the time complexity of Seller's algorithm for NNSM problem is related to the pattern length, but our algorithm does not if the pattern length is smaller than word size.

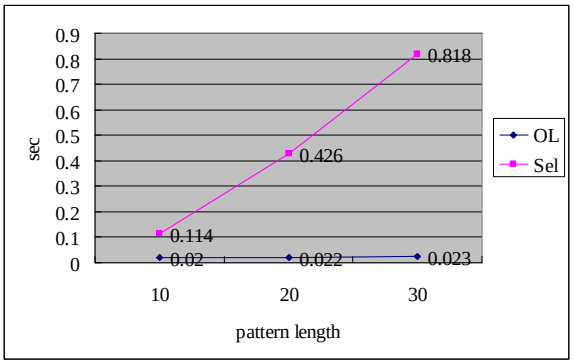

Figure 5. The experiment result for σ =4 and $|T|=10^5$ with $|P|$=10, 20 and 30

Figure 6 shows that the experiment result for σ = 4 and $|P|$ = 30 with $|T| = 10^5$, 2×10^5, 3×10^5, 4×10^5 and 5×10^5. This experiment shows the time complexity of Seller's algorithm for NNSM problem is related to the text length.

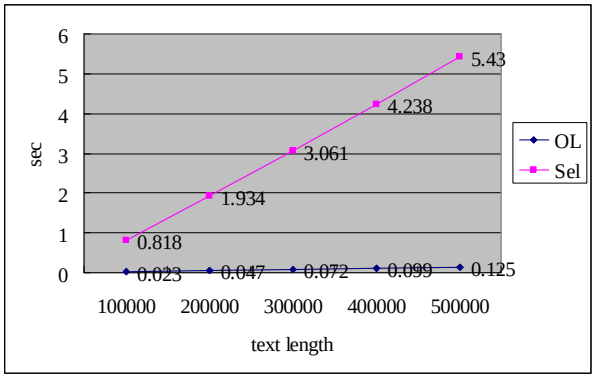

Figure 6 the experiment result for σ = 4 and $|P|$ = 30 with $|T| = 10^5$, 2×10^5, 3×10^5, 4×10^5 and 5×10^5.

In the experiment 3, we use the bioinformatics data of the DNA sequence of *Chlamydomonas* with length 103,840,115 for the text and the random DNA sequence with length 30. The result of Seller's algorithm is 1102.171 sec, and the result of ours is 18.232 sec.

6 Conclusions

This paper utilizes the bit-parallel approach to solve the nearest neighbor problem. The tradition way solves this problem by constructing the DP matrix in all locations of *T*. This kind of approach is greedy and inefficiency. Based on the bit-parallel approach and the filter methods, the proposed algorithm can speed up the tradition DP matrix approach to avoid exhaustive searching in the text. Our solutions for the NNSM and κ-NNSM problem are useful for applications in bioinformatics. The performance of the proposed scheme has been evaluated by performing a series of experiments. Overall, the experiment results have shown that the proposed algorithm outperforms Seller's algorithm in terms of the total elapsed time.

References

[1] H. Hyyro and G. Novarro. Bit-parallel witnesses and their application to approximate string matching, *Algorithmica*, 41(3):203-231, 2005.
[2] R. C. T. Lee. String matching algorithm, *Textbook*, 2010.
[3] G. Myers. A fast bit-vector algorithm for approximate string matching based on dynamic programming, *Journal of the ACM*, 46(3):395-415, 1999.
[4] W. J. Masek and M. S. Paterson. A faster algorithm computing string edit distances, *Journal of Computer and System Sciences*, 20:18-31, 1980.
[5] P. H. Sellers. The theory and computation of

evolutionary distances: pattern recognition, *Journal of Algorithms*, 1:359-373, 1980.

[6] E. Ukkonen. Finding approximate patterns in strings. *Journal of Algorithms*, 6:100-118, 1985.

[7] R. A. Wagner and M. J. Fischer. The string-to-string correction problem, Journal of the ACM 21, 1: 168–173, 1974.